

Assignment 31

SQL

1. Customer Insights

Q. Top 5 customers by total spending

```
SELECT c.customer_id, c.name, SUM(o.amount) AS total_spending
FROM customers c
JOIN orders o ON c.customer_id = o.customer_id
GROUP BY c.customer_id, c.name
ORDER BY total_spending DESC
LIMIT 5;
```

Q. Customers who have not placed any orders

```
SELECT c.customer_id, c.name
FROM customers c
LEFT JOIN orders o ON c.customer_id = o.customer_id
WHERE o.order_id IS NULL;
```

Q. Customers who placed more than 3 orders in a month

```
SELECT c.customer_id, c.name,
       DATE_FORMAT(o.order_date, '%Y-%m') AS order_month,
       COUNT(*) AS order_count
FROM customers c
JOIN orders o ON c.customer_id = o.customer_id
GROUP BY c.customer_id, c.name, order_month
HAVING order_count > 3
ORDER BY order_month;
```

Q. First order date for each customer

```
SELECT c.customer_id, c.name, MIN(o.order_date) AS first_order_date
```

```
FROM customers c
JOIN orders o ON c.customer_id = o.customer_id
GROUP BY c.customer_id, c.name
ORDER BY first_order_date;
```

Q. Customers whose average order value is above overall average

```
SELECT c.customer_id, c.name,
       AVG(o.amount) AS avg_order_value
FROM customers c
JOIN orders o ON c.customer_id = o.customer_id
GROUP BY c.customer_id, c.name
HAVING avg_order_value > (SELECT AVG(amount) FROM orders)
ORDER BY avg_order_value DESC;
```

2. Order Analysis

Q. Daily total revenue

```
SELECT order_date, SUM(amount) AS daily_revenue
FROM orders
GROUP BY order_date
ORDER BY order_date;
```

Q. Month with highest sales

```
SELECT DATE_FORMAT(order_date, '%Y-%m') AS sales_month,
       SUM(amount) AS total_sales
FROM orders
GROUP BY sales_month
ORDER BY total_sales DESC
LIMIT 1;
```

Q. Orders where amount > average order amount

```
SELECT order_id, customer_id, order_date, amount
```

```
FROM orders
WHERE amount > (SELECT AVG(amount) FROM orders)
ORDER BY amount DESC;
```

Q. Find duplicate orders (same customer, same date, same amount)

```
SELECT customer_id, order_date, amount,
       COUNT(*) AS duplicate_count
FROM orders
GROUP BY customer_id, order_date, amount
HAVING duplicate_count > 1
ORDER BY duplicate_count DESC;
```

Q. Running total of revenue by date

```
SELECT order_date,
       SUM(amount) AS daily_revenue,
       SUM(SUM(amount)) OVER (ORDER BY order_date) AS running_total
FROM orders
GROUP BY order_date
ORDER BY order_date;
```

3. Product Analysis

Q. Top 3 best-selling products by quantity

```
SELECT p.product_id, p.product_name,
       SUM(oi.quantity) AS total_qty_sold
FROM products p
JOIN order_items oi ON p.product_id = oi.product_id
GROUP BY p.product_id, p.product_name
ORDER BY total_qty_sold DESC
LIMIT 3;
```

Q. Products never ordered

```
SELECT p.product_id, p.product_name
FROM products p
LEFT JOIN order_items oi ON p.product_id = oi.product_id
WHERE oi.product_id IS NULL;
```

Q. Total revenue per product

```
SELECT p.product_id, p.product_name,
       SUM(p.price * oi.quantity) AS total_revenue
FROM products p
JOIN order_items oi ON p.product_id = oi.product_id
GROUP BY p.product_id, p.product_name
ORDER BY total_revenue DESC;
```

Q. Most popular category by quantity sold

```
SELECT p.category,
       SUM(oi.quantity) AS total_qty
FROM products p
JOIN order_items oi ON p.product_id = oi.product_id
GROUP BY p.category
ORDER BY total_qty DESC
LIMIT 1;
```

Q. Products with price above category average

```
SELECT p.product_id, p.product_name, p.category, p.price
FROM products p
WHERE p.price > (
    SELECT AVG(p2.price)
    FROM products p2
    WHERE p2.category = p.category
)
```

ORDER BY p.category, p.price DESC;

4. Joins & Aggregation

Q. Customer name with total number of orders

```
SELECT c.customer_id, c.name,  
       COUNT(o.order_id) AS total_orders  
FROM customers c  
LEFT JOIN orders o ON c.customer_id = o.customer_id  
GROUP BY c.customer_id, c.name  
ORDER BY total_orders DESC;
```

Q. Customers who ordered all products in a category

```
SELECT c.customer_id, c.name  
FROM customers c  
JOIN orders o ON c.customer_id = o.customer_id  
JOIN order_items oi ON o.order_id = oi.order_id  
JOIN products p ON oi.product_id = p.product_id  
WHERE p.category = 'Electronics'  
GROUP BY c.customer_id, c.name  
HAVING COUNT(DISTINCT p.product_id) = (  
    SELECT COUNT(*) FROM products WHERE category = 'Electronics'  
);
```

Q. Customers who only ordered from one category

```
SELECT c.customer_id, c.name,  
       COUNT(DISTINCT p.category) AS category_count  
FROM customers c  
JOIN orders o ON c.customer_id = o.customer_id  
JOIN order_items oi ON o.order_id = oi.order_id
```

```
JOIN products p ON oi.product_id = p.product_id  
GROUP BY c.customer_id, c.name  
HAVING category_count = 1;
```

Q. Most expensive product in each order

```
SELECT o.order_id, p.product_name, p.price  
FROM orders o  
JOIN order_items oi ON o.order_id = oi.order_id  
JOIN products p ON oi.product_id = p.product_id  
WHERE p.price = (  
    SELECT MAX(p2.price)  
    FROM order_items oi2  
    JOIN products p2 ON oi2.product_id = p2.product_id  
    WHERE oi2.order_id = o.order_id  
)  
ORDER BY o.order_id;
```

Q. Rank customers based on total spending

```
SELECT c.customer_id, c.name,  
    SUM(o.amount) AS total_spending,  
    RANK() OVER (ORDER BY SUM(o.amount) DESC) AS spending_rank  
FROM customers c  
JOIN orders o ON c.customer_id = o.customer_id  
GROUP BY c.customer_id, c.name  
ORDER BY spending_rank;
```

MongoDB

1. Aggregation Basics

Q. Calculate total revenue

```
db.orders.aggregate([
  {
    $group: {
      _id: null,
      total_revenue: { $sum: "$total_amount" }
    }
  }
]);
```

Q. Find average order value

```
db.orders.aggregate([
  {
    $group: {
      _id: null,
      avg_order_value: { $avg: "$total_amount" }
    }
  }
]);
```

Q. Count total orders

```
// Count all orders
db.orders.countDocuments({});

// Alternative using aggregate
db.orders.aggregate([
```

```
{ $count: "total_orders" }  
]);
```

Q. Find maximum order amount

```
db.orders.aggregate([  
  {  
    $group: {  
      _id: null,  
      max_order: { $max: "$total_amount" }  
    }  
  }  
]);
```

Q. Get minimum order value

```
db.orders.aggregate([  
  {  
    $group: {  
      _id: null,  
      min_order: { $min: "$total_amount" }  
    }  
  }  
]);
```

2. Grouping & Analytics

Q. Total revenue per customer

```
db.orders.aggregate([  
  {  
    $group: {
```



```
    _id: "$customer_id",
    total_revenue: { $sum: "$total_amount" }
  }
},
{ $sort: { total_revenue: -1 } }
]);
```

Q. Top 5 customers by spending

```
db.orders.aggregate([
  {
    $group: {
      _id: "$customer_id",
      total_spending: { $sum: "$total_amount" }
    }
  },
  { $sort: { total_spending: -1 } },
  { $limit: 5 }
]);
```

Q. Count orders per day

```
db.orders.aggregate([
  {
    $group: {
      _id: {
        $dateToString: {
          format: "%Y-%m-%d",
          date: { $toDate: "$order_date" }
        }
      }
    }
  }
]);
```

```
    },  
    order_count: { $sum: 1 }  
  }  
},  
{ $sort: { _id: 1 } }  
]);
```

Q. Find most frequent customer

```
db.orders.aggregate([  
  {  
    $group: {  
      _id: "$customer_id",  
      order_count: { $sum: 1 }  
    }  
  },  
  { $sort: { order_count: -1 } },  
  { $limit: 1 }  
]);
```

Q. Average order value per month

```
db.orders.aggregate([  
  {  
    $group: {  
      _id: { $substr: ["$order_date", 0, 7] },  
      avg_order_value: { $avg: "$total_amount" }  
    }  
  },  
  { $sort: { _id: 1 } }
```

```
]);
```

3. Array Handling

Q. Unwind items and calculate total quantity sold

```
db.orders.aggregate([  
  { $unwind: "$items" },  
  {  
    $group: {  
      _id: null,  
      total_quantity: { $sum: "$items.quantity" }  
    }  
  }  
]);
```

Q. Find most sold product

```
db.orders.aggregate([  
  { $unwind: "$items" },  
  {  
    $group: {  
      _id: "$items.product_id",  
      total_qty: { $sum: "$items.quantity" }  
    }  
  },  
  { $sort: { total_qty: -1 } },  
  { $limit: 1 }  
]);
```

Q. Get revenue per category

```
db.orders.aggregate([
```

```

{ $unwind: "$items" },
{
  $group: {
    _id: "$items.category",
    category_revenue: {
      $sum: { $multiply: ["$items.price", "$items.quantity"]}
    }
  }
},
{ $sort: { category_revenue: -1 }}
]);

```

Q. Find orders containing more than 2 items

```

db.orders.aggregate([
{
  $match: {
    $expr: { $gt: [{ $size: "$items" }, 2] }
  }
},
{
  $project: {
    customer_id: 1,
    order_date: 1,
    item_count: { $size: "$items" }
  }
}
]);

```

Q. Find products sold more than 10 times (total quantity)

```
db.orders.aggregate([
  { $unwind: "$items" },
  {
    $group: {
      _id: "$items.product_id",
      total_sold: { $sum: "$items.quantity" }
    }
  },
  { $match: { total_sold: { $gt: 10 } } },
  { $sort: { total_sold: -1 } }
]);
```

4. Filtering & Conditions

Q. Find orders above average value

```
db.orders.aggregate([
  {
    $facet: {
      avg_val: [{ $group: { _id: null, avg: { $avg: "$total_amount" } } }],
      all_orders: [{ $project: { customer_id:1, total_amount:1 } } ]
    }
  },
  { $unwind: "$avg_val" },
  { $unwind: "$all_orders" },
  {
    $match: {
      $expr: {
```

```

    $gt: ["$all_orders.total_amount", "$avg_val.avg"]
  }
}
}
]);

```

Q. Get orders in last 30 days

```

const thirtyDaysAgo = new Date();
thirtyDaysAgo.setDate(thirtyDaysAgo.getDate() - 30);

```

```

db.orders.find({
  order_date: {
    $gte: thirtyDaysAgo.toISOString().split("T")[0]
  }
});

```

Q. Find customers with at least 3 orders

```

db.orders.aggregate([
  {
    $group: {
      _id: "$customer_id",
      order_count: { $sum: 1 }
    }
  },
  { $match: { order_count: { $gte: 3 } } },
  { $sort: { order_count: -1 } }
]);

```

Q. Find orders containing a specific category (e.g., Electronics)

```
db.orders.find({
  items: {
    $elemMatch: { category: "Electronics" }
  }
});
```

Q. Get orders where total_amount != sum of items

```
db.orders.aggregate([
  { $unwind: "$items" },
  {
    $group: {
      _id: "$_id",
      stored_total: { $first: "$total_amount" },
      computed_total: {
        $sum: { $multiply: ["$items.price", "$items.quantity"] }
      }
    }
  },
  {
    $match: {
      $expr: { $ne: ["$stored_total", "$computed_total"] }
    }
  }
]);
```